📓 جزوه سیستم عامل - فصل ۷: مثال‌های همگامی

(بخش دوم: راه حل مانیتور برای فلاسفه و همگامی در ویندوز/لینوکس)

۱. راه حل مانیتوری مسئله فلاسفه خوش‌خوراک

(ادامه از بخش قبل)

در بخش قبل دیدیم که راه حل سمافوری برای مسئله فلاسفه ممکن است باعث **بن‌بست** (Deadlock) شود. حالا راه حل **مانیتور** را بررسی می‌کنیم که ایمن‌تر است.

ساختار مانیتور DiningPhilosophers

ما ۳ وضعیت برای هر فیلسوف تعریف می‌کنیم:

1. THINKING (فکر کردن)
2. HUNGRY (گرسنه - یعنی می‌خواهد چوبک بردارد ولی هنوز نتوانسته)
3. EATING (در حال خوردن)

شرط خوردن:

فیلسوف i فقط زمانی می‌تواند EATING شود که:

- خودش HUNGRY باشد.
- همسایه چپش EATING **نباشد**.
- همسایه راستش EATING **نباشد**.

کدهای مهم مانیتور:

الف) برداشتن چوبک (pickup):

وقتی فیلسوف گرسنه می‌شود این تابع را صدا می‌زند:

```
} void pickup(int i)
    state[i] = HUNGRY; // ۱. اعلام گرسنگی
    test(i);           // ۲. تلاش برای گرفتن چوبک‌ها
    if (state[i] != EATING) // ۳. اگر موفق نشد (چون همسایه می‌خورد)
        self[i].wait(); // ۴. منتظر بمان (خواب)
{
```

ب) گذاشتن چوبک (putdown):

وقتی غذا تمام شد:

```
} void putdown(int i)
    state[i] = THINKING; // ۱. تغییر وضعیت به فکر کردن

    // ۲. حالا که من دست از غذا کشیدم، شاید همسایه‌هایم بتوانند بخورند:
    test((i + 4) % 5);   // تست همسایه چپ
    test((i + 1) % 5);   // تست همسایه راست
{
```

ج) تابع تست (test):

این تابع بررسی می‌کند آیا شرایط برای غذا خوردن فیلسوف `i` مهیاست یا نه.

```
} void test(int i)
    if ((state[(i + 4) % 5] != EATING) // && همسایه چپ نمی‌خورد
        && (state[i] == HUNGRY)        // خودم گرسنه‌ام
        && ((state[(i + 1) % 5] != EATING) // همسایه راست نمی‌خورد { )

        state[i] = EATING; // وضعیت را به خوردن تغییر بده ;
        self[i].signal();  // اگر خواب بود، بیدارش کن ;
    }
{
```

نکته امتحانی: این روش تضمین می‌کند که **بن‌بست رخ نمی‌دهد**، اما تضمین نمی‌کند که **قحطی‌زدگی (Starvation)** رخ ندهد (ممکن است یک فیلسوف بدشانس همیشه بین دو همسایه پرخور گیر کند!).

۲. همگامی در ویندوز (Windows Synchronization)

ویندوز یک سیستم چندنخی (Multithreaded) و با قابلیت بلادرنگ (Real-time) است.

مدیریت در هسته (Kernel):

- **تک‌پردازنده (Single-processor):** برای حفاظت از منابع سراسری، موقتاً **وقفه‌ها (Interrupts) را غیرفعال می‌کند**.
- **چندپردازنده (Multi-processor):** از **قفل چرخشی (Spinlock)** استفاده می‌کند. (چون غیرفعال کردن وقفه در همه پردازنده‌ها سخت است).
    - **قانون مهم:** نخی که قفل چرخشی دارد، هرگز نباید **Preempt** (پس گرفته) شود.

اشیاء اعزام‌کننده (Dispatcher Objects):

ابزارهایی هستند که نخ‌ها برای همگامی خارج از هسته استفاده می‌کنند. دو وضعیت دارند:

1. **Signaled (سیگنال‌دار):** در دسترس است (مثل چراغ سبز). نخ مسدود نمی‌شود.
2. **Nonsignaled (بی‌سیگنال):** در دسترس نیست (مثل چراغ قرمز). نخ مسدود می‌شود.

انواع اشیاء:

- **Mutex:** فقط ۱ نخ را عبور می‌دهد (مالکیت انحصاری).
- **Semaphore:** چند نخ را عبور می‌دهد (شمارشی).
- **Event:** مثل متغیر شرط است (به نخ منتظر خبر می‌دهد که اتفاقی افتاده).
- **Timer:** وقتی زمان مشخصی گذشت، سیگنال می‌دهد.

اشیاء بخش بحرانی (Critical Section Objects):

- این‌ها قفل‌های **User-mode** هستند (بدون دخالت هسته).
- **مزیت:** سرعت بسیار بالا (چون سوئیچ به مد هسته ندارند).
- **عملکرد ترکیبی:** در سیستم چندپردازنده، ابتدا کمی از **Spinlock** استفاده می‌کند (می‌چرخد)، اگر قفل آزاد نشد، آن‌وقت سراغ هسته می‌رود و مسدود می‌شود. (چون اکثر قفل‌ها زود آزاد می‌شوند، این روش کارایی را عالی می‌کند).

۳. همگامی در لینوکس (Linux Synchronization)

ویژگی‌های هسته لینوکس:

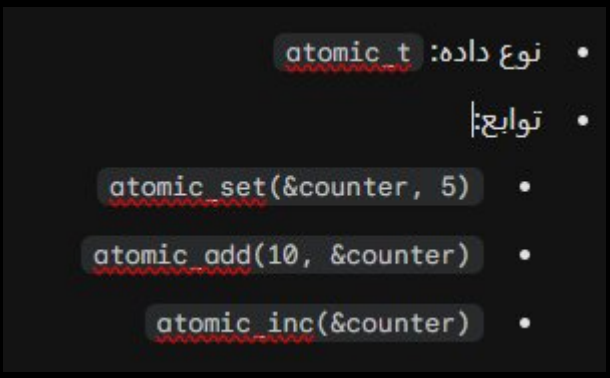
- قبل از نسخه 2.6: غیرقابل پس‌گیر (Non-preemptive).

- نسخه 2.6 به بعد: **کاملاً پس‌گیر (Fully Preemptive)**. (یعنی حتی وسط اجرای کد هسته هم می‌توان سوئیچ کرد).

ابزارهای همگامی در لینوکس:

۱. اعداد صحیح اتمیک (Atomic Integers)

ساده‌ترین روش همگامی. به جای قفل، از متغیرهای خاصی استفاده می‌کند که عملیات ریاضی روی آن‌ها اتمیک است.



- **مزیت:** بسیار سریع و بدون سربار قفل. مناسب برای شمارنده‌ها.

۲. قفل mutex_lock) Mutex)

برای حفاظت از بخش بحرانی در هسته. اگر قفل نباشد، وظیفه (Task) را به **خواب** می‌برد.

۳. قفل چرخشی (Spinlock) و مدیریت Preemption

لینوکس هم مثل ویندوز در چندپردازنده‌ها از Spinlock استفاده می‌کند.

نکته کلیدی: در تک‌پردازنده، Spinlock عملاً تبدیل به **غیرفعال کردن Preemption** می‌شود.

جدول رفتار Spinlock در لینوکس:

محیط	رفتار هسته
تک‌پردازنده	فقط **Preemption** را غیرفعال می‌کند (قفل واقعی نمی‌گیرد).
چندپردازنده	واقعاً **Spinlock** را می‌گیرد (قفل چرخشی).

مدیریت Preemption (شمارنده preempt_count):

لینوکس برای هر نخ یک متغیر preempt_count دارد:

- هر وقت قفلی می‌گیریم -> +1 می‌شود.
- هر وقت قفلی آزاد می‌کنیم -> -1 می‌شود.
- **قانون:** اگر preempt_count > 0 باشد، یعنی نخِ جاری قفل دارد، پس هسته حق ندارد آن را متوقف (Preempt) کند (چون خطرناک است). اگر 0 باشد، توقف مجاز است.

پایان بخش دوم فصل ۷